

Image Classification Model

Convolutional neural network built with Keras and TensorFlow

Program

CODE

```
# -----
# Step 1 - Create a new Python 3 notebook and install PyDrive
# (Google Colab only - not needed when the data is already local)
# -----
# !pip install PyDrive
# import os
# from pydrive.auth import GoogleAuth
# from pydrive.drive import GoogleDrive
# from google.colab import auth
# from oauth2client.client import GoogleCredentials

# -----
# Step 2 - Import the libraries needed for the model building phase
# -----
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D
from keras.utils import to_categorical, load_img, img_to_array
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from tqdm import tqdm

# -----
# STAGE A - Loading and preprocessing the image data
# Step 3 - Load the data and read all the training images
# -----
train = pd.read_csv('train.csv')
print(train.head())

# read all the training images, store them in a list and finally
# convert that list into a numpy array
# (grayscale images -> color_mode='grayscale'; for RGB use 'rgb')
train_image = []
for i in tqdm(range(train.shape[0])):
    img = load_img('train/' + train['id'][i].astype('str') + '.png',
                  target_size=(28, 28), color_mode='grayscale')
    img = img_to_array(img)
    img = img / 255
    train_image.append(img)
X = np.array(train_image)
print(X.shape)

# one-hot encode the target variable
y = train['label'].values
y = to_categorical(y)

# Step 4 - Creating a validation set from the training data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=42, test_size=0.2)
print(X_train.shape, X_test.shape)

# -----
# STAGE B - Defining the model's architecture
# Step 5 - 2 convolutional layers, 1 dense hidden layer, 1 output layer
# -----
model = Sequential()
model.add(keras.Input(shape=(28, 28, 1)))
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu'))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
```

```

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
model.summary()

# compile the model
model.compile(loss='categorical_crossentropy', optimizer='Adam',
              metrics=['accuracy'])

# -----
# STAGE C - Training the model
# Step 6 - train on the training images, validate on the validation set
# -----
model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test),
         verbose=2)

# -----
# STAGE D - Estimating the model's performance
# -----
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Validation loss: %.4f accuracy: %.4f" % (loss, acc))

# Step 7 - Making predictions
# load the test images and pre-process them like the training data
test = pd.read_csv('test.csv')
test_image = []
for i in tqdm(range(test.shape[0])):
    img = load_img('test/' + test['id'][i].astype('str') + '.png',
                  target_size=(28, 28), color_mode='grayscale')
    img = img_to_array(img)
    img = img / 255
    test_image.append(img)
test = np.array(test_image)

# making predictions
# (model.predict_classes() was removed - use argmax on model.predict())
prediction = np.argmax(model.predict(test, verbose=0), axis=1)

# creating submission file
sample = pd.read_csv('sample_submission_I5njJSF.csv')
sample['label'] = prediction
sample.to_csv('sample_cnn.csv', header=True, index=False)
print(sample.head())
print(sample['label'].value_counts().sort_index())

```

Program output

Output of the program above, shown stage by stage. The dataset has 60,000 training images and 10,000 test images (28x28 grayscale, 10 classes).

Stage A - Loading and preprocessing the image data

OUTPUT

```

100%|#####| 60000/60000 [00:06<00:00, 9590.22it/s]
  id label
0    1     9
1    2     0
2    3     0
3    4     3
4    5     0
(60000, 28, 28, 1)
(48000, 28, 28, 1) (12000, 28, 28, 1)

```

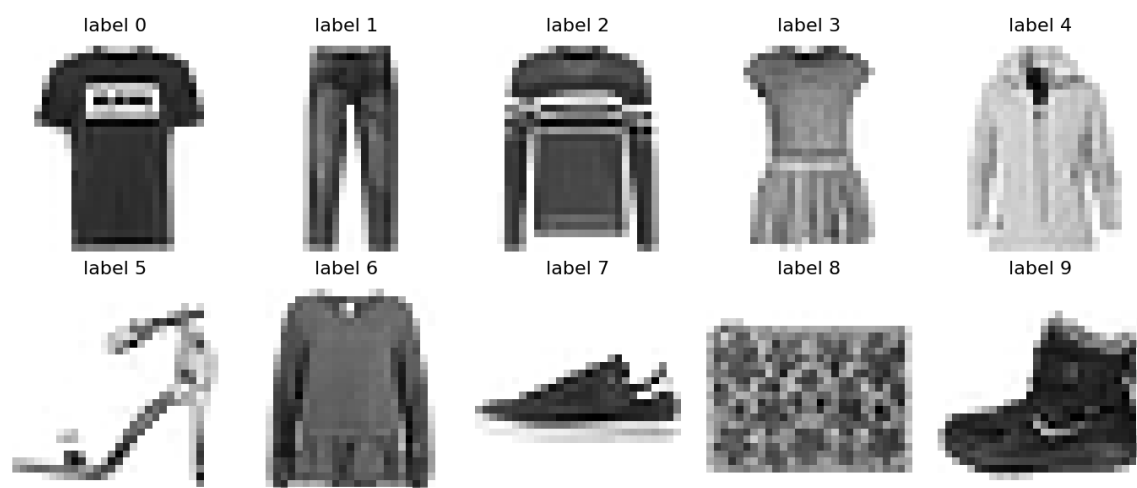


Figure. One training image from each of the 10 classes.

Stage B - Defining the model's architecture

OUTPUT

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
dropout (Dropout)	(None, 12, 12, 64)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1,179,776
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 1,199,882 (4.58 MB)
Trainable params: 1,199,882 (4.58 MB)
Non-trainable params: 0 (0.00 B)

Stage C - Training the model

OUTPUT

```
Epoch 1/10
1500/1500 - 40s - 27ms/step - accuracy: 0.8283 - loss: 0.4831 - val_accuracy: 0.8852 - val_loss: 0.3152
Epoch 2/10
1500/1500 - 41s - 27ms/step - accuracy: 0.8833 - loss: 0.3251 - val_accuracy: 0.9019 - val_loss: 0.2648
Epoch 3/10
1500/1500 - 42s - 28ms/step - accuracy: 0.9008 - loss: 0.2768 - val_accuracy: 0.9090 - val_loss: 0.2434
Epoch 4/10
1500/1500 - 40s - 27ms/step - accuracy: 0.9100 - loss: 0.2429 - val_accuracy: 0.9097 - val_loss: 0.2333
Epoch 5/10
1500/1500 - 41s - 28ms/step - accuracy: 0.9181 - loss: 0.2194 - val_accuracy: 0.9182 - val_loss: 0.2223
Epoch 6/10
1500/1500 - 41s - 28ms/step - accuracy: 0.9256 - loss: 0.1998 - val_accuracy: 0.9176 - val_loss: 0.2280
Epoch 7/10
1500/1500 - 41s - 27ms/step - accuracy: 0.9296 - loss: 0.1892 - val_accuracy: 0.9183 - val_loss: 0.2257
Epoch 8/10
```

```
1500/1500 - 40s - 27ms/step - accuracy: 0.9362 - loss: 0.1706 - val_accuracy: 0.9196 - val_loss: 0.2324
Epoch 9/10
1500/1500 - 41s - 27ms/step - accuracy: 0.9389 - loss: 0.1623 - val_accuracy: 0.9204 - val_loss: 0.2373
Epoch 10/10
1500/1500 - 42s - 28ms/step - accuracy: 0.9442 - loss: 0.1463 - val_accuracy: 0.9258 - val_loss: 0.2313
```

Stage D - Estimating the model's performance and making predictions

OUTPUT

```
Validation loss: 0.2313 accuracy: 0.9258
100%|#####| 10000/10000 [00:00<00:00, 10153.85it/s]
      id label
0 60001      9
1 60002      2
2 60003      1
3 60004      1
4 60005      6
label
0      969
1      987
2     1028
3     1009
4     1032
5      990
6      970
7     1007
8     1001
9     1007
Name: count, dtype: int64
```

Result

After 10 epochs the model reaches 94.4% accuracy on the training images and 92.6% on the validation set (loss 0.231). Validation accuracy stopped improving much after epoch 5, while training accuracy kept rising, which is the first sign of overfitting. The Dropout layers keep the gap small. The predictions for the 10,000 test images are saved to sample_cnn.csv.

Feed Forward Neural Network with Keras and TensorFlow

Handwritten digit classification on MNIST using SGD

Program

Run as: `python keras_mnist.py --output output/keras_mnist.png`

CODE

```
# import the necessary packages
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import matplotlib.pyplot as plt
import numpy as np
import argparse

# construct the argument parser and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-o", "--output", required=True,
                help="path to the output loss/accuracy plot")
args = vars(ap.parse_args())

# grab the MNIST dataset (if this is your first time using this
# dataset then the 11MB download may take a minute)
print("[INFO] accessing MNIST...")
((trainX, trainY), (testX, testY)) = mnist.load_data()

# each image in the MNIST dataset is represented as a 28x28x1
# image, but in order to apply a standard neural network we must
# first "flatten" the image to be simple list of 28x28=784 pixels
trainX = trainX.reshape((trainX.shape[0], 28 * 28 * 1))
testX = testX.reshape((testX.shape[0], 28 * 28 * 1))

# scale data to the range of [0, 1]
trainX = trainX.astype("float32") / 255.0
testX = testX.astype("float32") / 255.0

# convert the labels from integers to vectors (one-hot encoding)
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)

# define the 784-256-128-10 architecture using Keras
model = Sequential()
model.add(Input(shape=(784,)))
model.add(Dense(256, activation="sigmoid"))
model.add(Dense(128, activation="sigmoid"))
model.add(Dense(10, activation="softmax"))

# train the model using SGD
print("[INFO] training network...")
sgd = SGD(0.01)
model.compile(loss="categorical_crossentropy", optimizer=sgd,
              metrics=["accuracy"])
H = model.fit(trainX, trainY, validation_data=(testX, testY),
              epochs=100, batch_size=128, verbose=2)

# evaluate the network
print("[INFO] evaluating network...")
predictions = model.predict(testX, batch_size=128)
print(classification_report(testY.argmax(axis=1),
                           predictions.argmax(axis=1),
                           target_names=[str(x) for x in lb.classes_]))

# plot the training loss and accuracy
plt.style.use("ggplot")
```

```
plt.figure()
plt.plot(np.arange(0, 100), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 100), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 100), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 100), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.savefig(args["output"])
```

One-hot encoding of the labels

LabelBinarizer turns each integer label into a vector of length 10 with a single 1 at the position of the digit. For example, the label 3 becomes [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. The encoding of every digit is shown below.

OUTPUT

```
0: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
1: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
2: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
3: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
4: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
5: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
6: [0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
7: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
8: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]
9: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
```

Program output

Console output of the program above. Epochs 4 to 97 are truncated for brevity; the training ran for all 100 epochs.

OUTPUT

```
[INFO] accessing MNIST...
[INFO] training network...
Epoch 1/100
469/469 - 2s - 4ms/step - accuracy: 0.2113 - loss: 2.2720 - val_accuracy: 0.2683 - val_loss: 2.2356
Epoch 2/100
469/469 - 2s - 4ms/step - accuracy: 0.3688 - loss: 2.2004 - val_accuracy: 0.4606 - val_loss: 2.1582
Epoch 3/100
469/469 - 1s - 2ms/step - accuracy: 0.4826 - loss: 2.1094 - val_accuracy: 0.4517 - val_loss: 2.0485
[... epochs 4 to 97 truncated for brevity ...]
Epoch 98/100
469/469 - 1s - 2ms/step - accuracy: 0.9215 - loss: 0.2719 - val_accuracy: 0.9231 - val_loss: 0.2658
Epoch 99/100
469/469 - 1s - 3ms/step - accuracy: 0.9215 - loss: 0.2710 - val_accuracy: 0.9240 - val_loss: 0.2650
Epoch 100/100
469/469 - 1s - 3ms/step - accuracy: 0.9218 - loss: 0.2701 - val_accuracy: 0.9234 - val_loss: 0.2643
[INFO] evaluating network...

79/79   0s 1ms/step
        precision    recall  f1-score   support

         0           0.94       0.98       0.96        980
         1           0.97       0.97       0.97       1135
         2           0.92       0.91       0.91       1032
         3           0.91       0.90       0.91       1010
         4           0.92       0.93       0.93        982
         5           0.90       0.86       0.88        892
         6           0.93       0.94       0.94        958
         7           0.94       0.92       0.93       1028
         8           0.89       0.90       0.89        974
         9           0.90       0.91       0.91       1009

    accuracy
macro avg           0.92       0.92       0.92       10000
weighted avg        0.92       0.92       0.92       10000
```

Training loss and accuracy

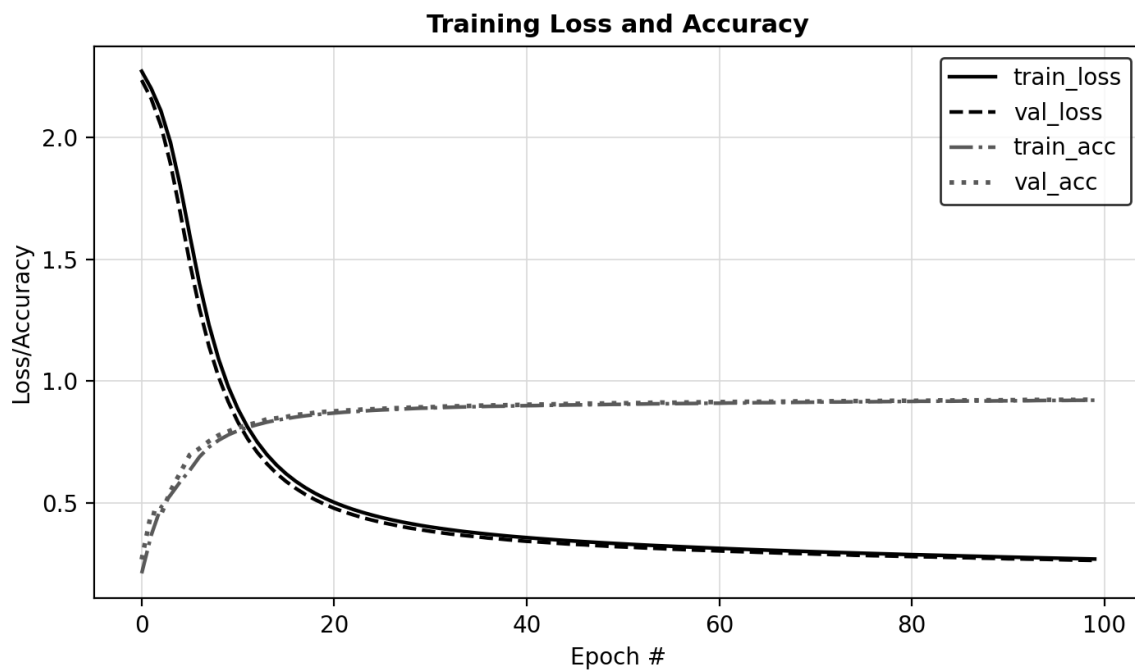


Figure. Training and validation loss and accuracy over 100 epochs. Loss falls steadily while accuracy rises, and the training and validation curves stay close together, so there is no sign of overfitting.

Result

After 100 epochs of SGD (learning rate 0.01, batch size 128) the 784-256-128-10 network reaches about 92% accuracy on the 10,000 MNIST test images. Digits 0 and 1 are recognised best and digit 5 is the hardest (F1 of 0.88).

Deep Learning Sample Programs

Source code with output: TensorFlow, Keras, Theano and PyTorch

1. TensorFlow

Fits a straight line $y = 0.2x + 0.2$ to 100 random points by minimizing mean squared error with the Adam optimizer.

CODE

```
# !pip install tensorflow
# import
import tensorflow as tf
import numpy as np

a = tf.constant(15)
b = tf.constant(20)
print(a + b)

# input
np.random.seed(0) # fixed seed -> reproducible output
x = np.random.rand(100).astype(np.float32)
print(x)

# output - observed
y = x * 0.2 + 0.2

# Weight
W = tf.Variable(tf.random.normal([1]))
# bias
b = tf.Variable(tf.zeros([1]))

# Create a function for MSE - mean squared error
def mse_loss():
    ypred = W * x + b
    loss = tf.reduce_mean(tf.square(ypred - y))
    return loss

# Optimizer
optimizer = tf.keras.optimizers.Adam()

# Iterations
for step in range(5000):
    with tf.GradientTape() as tape:
        loss = mse_loss()
    grads = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(grads, [W, b]))
    if step % 500 == 0:
        print(step, W.numpy(), b.numpy())
```

OUTPUT

```
tf.Tensor(35, shape=(), dtype=int32)
[0.5488135 0.71518934 0.60276335 0.5448832 0.4236548 0.6458941
 0.4375872 0.891773 0.96366274 0.3834415 0.79172504 0.5288949
 0.56804454 0.92559665 0.07103606 0.0871293 0.0202184 0.83261985
 0.77815676 0.87001216 0.9786183 0.7991586 0.46147937 0.7805292
 0.11827443 0.639921 0.14335328 0.9446689 0.5218483 0.41466194
 0.2645556 0.7742337 0.45615032 0.56843394 0.0187898 0.6176355
 0.6120957 0.616934 0.94374806 0.6818203 0.3595079 0.43703195
 0.6976312 0.06022547 0.6667667 0.67063785 0.21038257 0.12892629
 0.31542835 0.36371076 0.57019675 0.43860152 0.9883738 0.10204481
 0.20887676 0.16130951 0.6531083 0.2532916 0.46631077 0.2444256
 0.15896958 0.11037514 0.6563296 0.13818295 0.19658236 0.36872518
 0.82099324 0.09710128 0.8379449 0.09609841 0.97645944 0.4686512
 0.9767611 0.6048455 0.7392636 0.03918779 0.28280696 0.12019656
 0.2961402 0.11872772 0.31798318 0.41426298 0.06414749 0.6924721
 0.56660146 0.2653895 0.5232481 0.09394051 0.5759465 0.9292962
 0.31856894 0.6674104 0.13179787 0.7163272 0.2894061 0.18319136
 0.5865129 0.02010755 0.82894003 0.00469548]
0 [0.65013576] [-0.00099987]
500 [0.2964534] [0.1540868]
1000 [0.20662601] [0.19684625]
1500 [0.20010924] [0.19994801]
```

```

2000 [0.2000005] [0.19999976]
2500 [0.20000044] [0.1999998]
3000 [0.20000035] [0.19999984]
3500 [0.20000026] [0.19999988]
4000 [0.2000002] [0.1999999]
4500 [0.20000015] [0.19999994]

```

2. Keras

A small feed-forward network (8 - 12 - 8 - 1) that predicts diabetes onset from the Pima Indians dataset (768 rows).

CODE

```

# !pip install tensorflow
# Step 1 - Load the dataset
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

## INPUT Variables ##
# x1 - Number of times pregnant
# x2 - plasma glucose
# x3 - diastolic blood pressure
# x4 - Triceps skin fold thickness
# x5 - 2-hour serum insulin
# x6 - bmi
# x7 - diabetes pedigree function
# x8 - age (yrs)
## Output Variable ##
# Class Variable - 0 or 1

dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
print(dataset)

#[:,:] - first : is range of rows and second : is columns
# [start:end] - begins at start, ends at end-1
x = dataset[:, 0:8]
print(type(x))
print(x.shape)
y = dataset[:, 8]
print(y)

# Step 2 - Creating or define the Keras Model
# Sequential Model
# Layer1 -> Layer2 -> Layer3
model = Sequential()
# The model expects row of data with 8 variables
model.add(Input(shape=(8,)))
# 12 = nodes
model.add(Dense(12, activation='relu'))
# Hidden Layer
# 8 = nodes
model.add(Dense(8, activation='relu'))
# Output layer
model.add(Dense(1, activation='sigmoid'))

# Step 3 - Compile the Keras model
# loss (error)
# optimizer (adam)
# metrics = accuracy
model.compile(loss='binary_crossentropy', optimizer='adam',
              metrics=['accuracy'])

# Step 4 - Fit / Train the model
# 1 - Epochs - number of iterations / passes
# 2 - Batch - sample data
model.fit(x, y, epochs=150, batch_size=10, verbose=2)

# Step 5 - evaluate the model
model.evaluate(x, y)

```

OUTPUT

```

[[ 6.    148.    72.    ...  0.627 50.    1.   ]
 [ 1.     85.    66.    ...  0.351 31.    0.   ]

```

```
[ 8.    183.    64.    ...  0.672 32.    1.    ]
...
[ 5.    121.    72.    ...  0.245 30.    0.    ]
[ 1.    126.    60.    ...  0.349 47.    1.    ]
[ 1.    93.    70.    ...  0.315 23.    0.    ]]
<class 'numpy.ndarray'>
(768, 8)
[1. 0. 1. 0. 1. 0. 1. 0. 1. 1. 0. 1. 0. 1. 1. 1. 1. 0. 1. 0. 0. 1. 1.
 1. 1. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 1. 1. 1. 0. 0. 0. 1. 0. 1. 0. 0.
 1. 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 1. 0. 1. 0. 0. 0. 1. 0.
 1. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0.
 [... output truncated for brevity ...]

Epoch 1/150
77/77 - 1s - 11ms/step - accuracy: 0.5872 - loss: 2.2884
Epoch 2/150
77/77 - 0s - 1ms/step - accuracy: 0.4674 - loss: 0.8336
Epoch 3/150
77/77 - 0s - 2ms/step - accuracy: 0.4544 - loss: 0.7419
[... output truncated for brevity ...]
Epoch 148/150
77/77 - 0s - 2ms/step - accuracy: 0.6914 - loss: 0.5464
Epoch 149/150
77/77 - 0s - 2ms/step - accuracy: 0.6953 - loss: 0.5487
Epoch 150/150
77/77 - 0s - 2ms/step - accuracy: 0.6966 - loss: 0.5494

24/24 - 0s - 1ms/step - accuracy: 0.6940 - loss: 0.5422
```

3. Theano

Builds symbolic scalar and matrix expressions, compiles them into functions and evaluates them.

CODE

```
# Theano
# Mathematical Operations
# Arrays - Multi-dimensional
# Installation
# !pip install theano
import theano
from theano import *
import theano.tensor as T
import numpy as np
import pandas as pd
from theano import function

# scalar variables
v1 = T.dscalar()
v2 = T.scalar()

# subtraction
sres = v1 - v2
# add
ares = v1 + v2

# convert the results into functions
calcsres = theano.function([v1, v2], sres)
calcares = theano.function([v1, v2], ares)
calcares(12, 23)
calcsres(13, 12)

x = T.dmatrix('x')
y = T.dmatrix('y')
# addition
z = x + y
func = function([x, y], z)
m1 = [
    [1, 2],
    [3, 4]
]
m2 = [
    [4, 5],
    [6, 7]
```

```
]
func(m1, m2)
# element wise sum
# 00 -> [1+4] -> 5
```

OUTPUT

```
array(35.)
array(1.)
array([[ 5.,  7.],
       [ 9., 11.]])
```

4. PyTorch

Introduces tensors: creating them from data, NumPy arrays and shapes, then joining, multiplying, in-place updates

CODE

```
# Two major objectives of PyTorch
# 1. Replacement of Numpy to use the power of GPUs and other accelerators
# 2. Automatic Differentiation Library helps in implementation of
#    Neural Networks
# Feed Forward --> first stage
# Back Propagation --> second stage (differentiation)
# Tensors - Special Data Structures
# similar to an array, matrices
# similar to Numpy ndarrays
import torch
import numpy as np

torch.manual_seed(0)                                # fixed seed -> reproducible output

# Tensor Initialization
### multiple ways
### 1 - using data
data = [
    [1, 2],
    [3, 4]
]
x_data = torch.tensor(data)
print(type(x_data))

### 2 - using numpy array
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
print(x_np)
print(type(x_np))

### 3 - using another tensor
x_ones = torch.ones_like(x_data)
print("One Tensor: \n", x_ones)
x_rand = torch.rand_like(x_data, dtype=torch.float)
print(x_rand)

#### more ways to create tensors
shape = (2, 3)
random_tensor = torch.rand(shape)
print(random_tensor)
print(type(random_tensor))
ones_tensor = torch.ones(shape)
print(ones_tensor)
print(type(ones_tensor))
zeros_tensor = torch.zeros(shape)
print(zeros_tensor)
print(type(zeros_tensor))

tensor = torch.rand(3, 4)
print(tensor)
print(tensor.shape)
print(tensor.dtype)
print(tensor.device)

# Tensor Operations
if torch.cuda.is_available():
    tensor = tensor.to('cuda')
```

```

print("Device tensor is stored in ", tensor.device)

# Indexing, Slicing
tensor = torch.ones(4, 4)
print(tensor)
tensor1 = torch.zeros(4, 4)
print(tensor1)
tensor2 = torch.cat([tensor, tensor1])
print(tensor2)

# Multiply Operation
print(tensor.mul(tensor1))
print(tensor * tensor1)
print(tensor.T)

# inplace - change the original tensor
tensor.add_(3)
print(tensor)

# from tensor to numpy
t = torch.ones(5)
print(t)
n = t.numpy()
print(n)
print(type(n))

```

OUTPUT

```

<class 'torch.Tensor'>
tensor([[1, 2],
        [3, 4]])
<class 'torch.Tensor'>
One Tensor:
  tensor([[1, 1],
          [1, 1]])
tensor([[0.4963, 0.7682],
        [0.0885, 0.1320]])
tensor([[0.3074, 0.6341, 0.4901],
        [0.8964, 0.4556, 0.6323]])
<class 'torch.Tensor'>
tensor([[1., 1., 1.],
        [1., 1., 1.]])
<class 'torch.Tensor'>
tensor([[0., 0., 0.],
        [0., 0., 0.]])
<class 'torch.Tensor'>
tensor([[0.3489, 0.4017, 0.0223, 0.1689],
        [0.2939, 0.5185, 0.6977, 0.8000],
        [0.1610, 0.2823, 0.6816, 0.9152]])
torch.Size([3, 4])
torch.float32
cpu
Device tensor is stored in cpu
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.]])

```

```
    [0., 0., 0., 0.],
    [0., 0., 0., 0.])
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
tensor([[4., 4., 4., 4.],
        [4., 4., 4., 4.],
        [4., 4., 4., 4.],
        [4., 4., 4., 4.]])
tensor([1., 1., 1., 1., 1.])
[1. 1. 1. 1. 1.]
<class 'numpy.ndarray'>
```